INDIA.RUNS

Build what next India runs on

Team Name : **Biscuit**

Team Leader Name : **Vinayak Sonthalia**

Problem Statement : **Track 01 — The Data & AI Challenge · Intelligent Candidate Discovery**

SOLUTION OVERVIEW

A deterministic, explainable ranking engine

Biscuit ranks the top 100 candidates from a 100,000-profile pool for a Senior AI Engineer role — CPU-only, zero-dependency, fully reproducible.

What it does

- Streams **100,000 profiles** and scores each on **six JD-derived dimensions**.
- Hard-eliminates **impossible "honeypot" profiles**; down-weights borderline ones.
- Emits a top-100 shortlist with a **unique, fact-grounded reason** per candidate.

What makes it different

- **Reasons about JD intent**, not keyword counts — catches keyword stuffers.
- **Penalizes services-only careers**; rewards product-company ML builders.
- **Down-weights "dead leads"** (inactive / unresponsive) per the JD.

100K profiles candidates.jsonl → **Filter** honeypots → **Score** 6 dimensions → **Rank + Reason** top 100 → **Biscuit.csv** submission

READING THE JOB DESCRIPTION

What the role *says* vs. what it *means*

Senior AI Engineer (Founding Team) · Redrob AI · Pune/Noida, India · 5–9 yrs (ideal 6–8).

Must have

- Embeddings-based **retrieval**
- **Vector DB / hybrid search**
- Strong **Python**
- Ranking eval — **NDCG / MRR / MAP**

Explicitly not wanted

- **Services-only** careers (TCS, Infosys...)
- **CV / speech**-primary, no NLP/IR
- **Keyword stuffers** (non-tech titles)
- Pure research · title-chasers

Reading between the lines

- A strong fit may use **no buzzwords** — but shipped a real recsys at a product company.
- A perfect skill list with a **"Marketing Manager"** title is **not** a fit.

Biscuit encodes this directly: a **career-relevance** signal (product vs. services, shipped-system language), a **skills gate** that requires genuine ML/IR relevance, and **behavioral signals** that measure whether a candidate is actually reachable.

RANKING METHODOLOGY

Six weighted dimensions, then two guardrails

DIMENSION	WEIGHT	MEASURES
Career relevance	30%	product vs. services, ML titles
Skills match	25%	core IR/NLP, proficiency-weighted
Experience band	15%	flat peak across 6–8 yrs
Behavioral	15%	activity, response, availability
Location	10%	India / notice / work mode
Education	5%	institution tier, field

Two guardrails after the weighted sum

Σ weighted score

↓

Suspicion multiplier
× (1 − penalty) for borderline data

↓

Skills gate
<0.05 → ×0.15 · <0.15 → ×0.35 · <0.25 → ×0.60

↓

final score

The skills gate is the key idea: relevance is a **precondition**, not just one term in a sum — so a non-technical profile with great location and engagement can never outrank a genuine ML engineer.

DATA INTEGRITY

Catching the "perfect-on-paper" traps

The dataset embeds ~80 honeypots — subtly impossible profiles forced to relevance tier 0 in the hidden ground truth. Biscuit hard-eliminates the 42 that are *provably* impossible; the rest score low and never reach the top 100.

Honeypot decision flow

≥3 "expert" skills with 0 months used?

↓ no

a single job longer than the person's total experience?

↓ no

stated duration > calendar span by >12 mo?

↓ no

scored (minus any suspicion penalty)

Rules 1 & 2 catch all 42 on this dataset; rule 3 is an impossibility guard that does not trigger here.

42
hard-eliminated

0
honeypots in the top 100

Conservative by design

- All 42 fire only on a **logical impossibility** (e.g. a single job longer than the person's total experience) — impossible by construction, not by label.
- Softer inconsistencies become a graded **suspicion penalty**, not elimination.
- No special-casing beyond impossibility — scoring naturally avoids the rest.

EXPLAINABILITY

A fact-grounded reason for every pick

No LLM, no generative text — every justification is composed from real profile fields, so it is deterministic and free of invented skills or employers.

How the reasoning is built

- **No fabricated entities:** years, titles, skills, and companies all come straight from the profile — there is no LLM in the loop.
- **Deterministic variation:** a per-candidate `MD5(candidate_id)` seed drives sentence templates — same candidate, same text every run; different candidates read differently.
- **Honest tone:** strong facts lead; gaps (long notice, junior, low engagement) are stated so tone matches rank.

```
rank 1 · score 0.989
Notably, Brings Learning To Rank, Qlora experience. 7 years of
experience, currently Senior Machine Learning Engineer at Zomato.
```

```
rank 3 · score 0.9766
Notably, Built retrieval-based systems at Vedantu. Previously
Applied ML Engineer at Vedantu; Previously NLP Engineer at Haptik.
```

Verbatim from `Biscuit.csv` — 100 unique reasoning strings, no templating repetition.

SYSTEM ARCHITECTURE

A transparent, modular pipeline

loader.py stream JSONL → **filters.py** honeypot + suspicion → **scorer.py** 6-dim weighted → **ranker.py** sort + reasoning → **Biscuit.csv** top 100

100,000 loaded → 42 honeypots removed → 99,958 scored → top 100 written · all parameters in `config.py`

- **loader.py**
  Memory-efficient streaming JSONL/JSON reader.

- **scorer.py**
  Six normalized dimensions + skills gate + suspicion multiplier.

- **ranker.py**
  Deterministic sort (↓score, ↑id) and reasoning builder.

Design philosophy: simple, transparent, deterministic Python over heavy neural models — CPU-only, well within budget, network-free, with byte-identical output across runs.

VALIDATION & METHODOLOGY

Validated by method, not by guessing

With no public leaderboard and hidden labels, the ranking was validated by recomputable checks over the full pool — not by submitting variants.

Top-100 health

- **0** honeypots · **0** services-only · **0** ghost profiles (<10% response)
- Median **6.45 years** experience — squarely in band
- Median **5** core JD skills per candidate
- Every top-10 shows concrete **shipped** retrieval / ranking / recsys work

Whole-pool calibration

- **Whole-pool ordering:** top 100 are ML/IR engineers; by rank ~500 titles turn non-technical (Business Analyst, Project Manager); deeper still, unrelated roles (Mechanical Engineer, DevOps, Graphic Designer).
- **Honeypot audit:** all 42 hard-removals are logical impossibilities; **0** reach the top 100.
- **Variants tested:** a TF-IDF semantic hybrid and a skill-synonym expansion were prototyped; without the hidden labels neither could be shown to beat the transparent heuristic baseline, which was kept.

The discipline matters as much as the result: a measured latency-quality trade-off, with variants rejected when they did not improve the top of the ranking.

RESULTS & PERFORMANCE

Fast, frugal, and fully reproducible

<30s
full 100K run · ~10s in Docker

~2 GB
peak RAM (limit: 16 GB)

0
third-party dependencies

Compute compliance (Stage 3)

CONSTRAINT	LIMIT	BISCUIT
Runtime	≤ 5 min	`< 30 s`
Memory	≤ 16 GB	`~2 GB`
Compute	CPU only	`CPU` ✓
Network	off	`none` ✓
Pre-compute	—	`none` ✓

Output quality & reproducibility

- **Deterministic:** byte-identical CSV across runs (MD5-seeded, not salted `hash()`).
- **Spec-valid:** exactly 100 rows, passes the official format validator.
- **Score range** 0.8855 → 0.989 — the model differentiates, not flat.
- No GPU, no LLM calls; ships with a Dockerfile for a sealed, network-free run.

Submission Assets

- **Links & deliverables**
  - **GitHub repository**
    `github.com/vinayaksonthalia/biscuit-candidate-ranker`
  - **Ranked output** — `Biscuit.csv` · top-100 with fact-grounded reasoning.
  - **Colab sandbox** — runs the pipeline end-to-end on a sample, with format & determinism checks.
  - **Presentation deck** — this document (`docs/Biscuit_Submission_Deck.pdf`).

- **Reproduce in one command**

```
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

Pure Python standard library · CPU-only · no network · <30 s on the full 100K pool. The uploaded **Biscuit.csv** is a byte-identical copy of this output.

<30s
full 100K run · limit 5 min

~2 GB
peak RAM · limit 16 GB

0
dependencies · GPU · network

INDIA.RUNS

Build what next India runs on

THANK YOU